PATENT-STYLE TECHNICAL DISCLOSURE

A System and Method for Object-Facing Projection Programming in JoeLang

Joe Team Working Draft — Version 0.9

Inventive subject matter. This disclosure specifies an object-facing programming language, a reversible source-projection process, a compiler architecture, and an online console model collectively designated JoeLang. It is an internal, creative, patent-style document built as a parody at the beginning of a wider Joe-universe technical record; it is not a legal filing or a claim of a real-world invention.

Source basis. The proposal extends the published Joelib archive: the Joe Wiki; Joeglish grammar and translation work; the Joeriodic Table of Joelements; Joekemon records; *The Great Joetsby*; the *Joelet Speech*; and JOESII art. The authorship attributions supplied for this working draft are recorded in the Sources and Attribution section.

Prepared for the Joe Team

November 7, 2026

Abstract

JoeLangis a source language for programming interactive systems through named objects, called Joebjects, their state, their methods, and their messages. A JoeLangprogram is projected into a host representation, initially JavaScript, without changing the intended observable behavior of the program. The disclosure defines: (i) a lexical projection dictionary patterned after the reversible English–Joeglish translation architecture; (ii) a constrained object-facing surface grammar; (iii) a Joebject normalization procedure; (iv) a receiver-first message-routing model; (v) a diagnostic system that reports projection and state errors in Joe vocabulary; and (vi) a web-console architecture for authoring, projecting, and executing programs.

The invention uses an important distinction established across Joelib: a familiar underlying system can remain stable while its Joe-facing representation changes. The Joelements material expressly retains conventional chemical symbols while presenting Joe names, and the translator documents a deterministic, one-to-one transformation with a verified round trip.[6, 3, 4] JoeLangapplies that design lesson to programming: stable JavaScript execution underneath, but a readable, object-facing Joe surface above.

Contents

Abstract 1

1 Field of the Disclosure 4

2 Background and Discovery Record 5

3 Source Record and Attribution 6

4 Definitions 7

5 System Overview 8

6 Core Language Rules 9

6.1 The Joebject rule . . . 9
6.2 The receiver rule . . . 9
6.3 The state rule . . . 9
6.4 The message rule . . . 9
6.5 The projection rule . . . 9

7 Normative Grammar 10

8 Central Algorithms 11

8.1 Algorithm A: lexical projection . . . 11
8.2 Algorithm B: Joebject normalization . . . 11
8.3 Algorithm C: receiver-first message routing . . . 11
8.4 Algorithm D: safe host emission . . . 11

9 Compiler and Console Embodiment 12

10 Library and Archive Model 13

11 Diagnostics, Safety, and Determinism 14

11.1 Diagnostic classes . . . 14
11.2 Safety profile . . . 14
11.3 Determinism profile . . . 14
11.4 Accessibility profile . . . 14

12 Illustrative Claims **15**

13 Conclusion and Use Notice **16**

Sources and Attribution **17**

1 Field of the Disclosure

This disclosure concerns programming-language design, source-to-source compilation, object-oriented and object-facing programming, interactive development environments, deterministic lexical transformation, and diagnostics. More specifically, it concerns a language in which a programmer states behavior through a JOEBJECTrather than beginning with disconnected procedures or raw host-language structures.

The term "object-facing" means that the primary unit encountered by a reader is an entity with a name, state, behavior, and messages. The language therefore asks: *which Joebject receives this request?* before asking: *which low-level operation will execute?* This ordering is both a readability discipline and a compiler constraint.

The disclosed system has four layers:

1. a **source layer**, written in JOELANGkeywords and Joeglish-compatible identifiers;
2. a **projection layer**, which tokenizes, validates, and maps source to an intermediate representation;
3. a **execution layer**, which emits JavaScript or an equivalent host representation; and
4. a **console layer**, which displays the source, projection, result, and diagnostics together.

2 Background and Discovery Record

The Joelib repository describes the Joe universe as a projection into the human universe and identifies Joe simultaneously with language, units, chemical elements, and other categories.[1] The accompanying wiki records an initial observation of Joe as a possible unit of length, identifies Joeglish as a spoken language, records a 118-member Joeriodic table, preserves literature and speech, and notes that a complete account of Joe mathematics or physics has not yet been found.[2]

The present disclosure treats that final gap as a design opportunity. If the archive supports a language, a translation discipline, a material naming system, cultural records, visual output, and indexed creature records, then a computational notation can be specified without contradicting the source archive. The discovery record therefore describes the design inference used by the Joe Team: existing Joe materials provide the semantic conventions from which a computational field manual can be reconstructed.

Three observations motivated the reconstruction.

1. The Joeglish translator describes a deterministic bidirectional mapping, including preservation of capitalization and successful round-trip verification.[3, 5]
2. The Joelements project presents 118 familiar elements under Joe names while keeping the standard chemical symbols and formulas meaningful.[6]
3. The archive is organized as a collection of studies, literature, visual signals, and regional creature lists rather than as a single closed work.[12, 7, 8]

Together, these observations support a language that changes a program's presentation while keeping an underlying execution substrate inspectable.

3 Source Record and Attribution

This document uses Joelib as its sole source family. The following attributions are supplied by the Joe Team for this disclosure and are retained here to make the design lineage explicit.

Contributor	**Archive materials**	**Design relevance to JoeLang**
Derek Li	Joelements; *The Great Joetsby*; Joe Wiki	Stable foundations with changed names; archive and discovery framing; vocabulary evidence.
Joe Jin (the central figure himself)	Joeglish Grammar	Joeglish lexical basis and grammatical discipline.
Wayne Xu	Joeglish Translator; JOESII; JOESII art	Deterministic translation, round-trip behavior, console-like visual treatment, and signal presentation.
Matthew DeRoma	Joekemons	Regional cataloguing, names, and package-registry structure.
Emilia Marcucci	*Joelet Speech*	Expressive, performative language evidence and the "speech as message" metaphor.

The repository README indexes the linked Joeglish Grammar, *The Great Joetsby*, the Joelements sheet, Joekemon collection, Joelet Speech, JOESII art, and Joegle Translate.[1] The source code and datasets additionally make several operational details inspectable: the translation engine separates words from punctuation, the generator rejects collisions, and the Joelements interface filters a complete element dataset while maintaining the original symbols.[4, 5, 6]

4 Definitions

For the purposes of this disclosure, the following terms have the meanings below.

Joe token A reserved keyword, identifier convention, literal convention, or diagnostic label belonging to JoeLang.

Joebject A named runtime entity containing declared state and declared behavior. A Joebject is the default receiver of a method call written within its body.

Projection A deterministic transformation from JoeLangsource to an intermediate representation and then to host code.

Host The execution environment that receives the projected result. Version 0.9 uses JavaScript as the normative host.

Receiver The Joebject or other value immediately to the left of a message operator or method invocation.

Message A request for behavior sent to a receiver, with zero or more ordered arguments.

State Named data declared by a Joebject with the `has` form and held for that Joebject.

Joelement A standard-library module whose stable host API is wrapped in a Joe-facing name and documentation card.

Round trip The condition in which a reversible lexical projection and its inverse recover the intended source-token sequence, excluding standardized whitespace.

These terms intentionally distinguish a source spelling from a runtime meaning. For example, a Joelement name may change in the source interface while its host-level implementation remains a stable JavaScript module, just as the Joelements data uses original chemical symbols as disambiguating anchors.[6]

5 System Overview

Figure-free system architecture is described as the following ordered path:

1. The author writes a JoeLangdocument in the online console.
2. A lexer preserves comments and literal text while classifying source words, punctuation, and structural delimiters.
3. A parser recognizes Joebject declarations, state declarations, behaviors, message sends, conditions, and imports.
4. A normalizer converts parsed Joebjects into a canonical object graph.
5. A lexical projector maps eligible surface identifiers by a validated Joe dictionary, while reserved keywords remain fixed.
6. A code generator emits JavaScript classes, objects, imports, and calls.
7. A sandbox executes the generated JavaScript and returns output, values, or diagnostics to the console.

The console must show both the authored source and generated host code. This is a non-negotiable transparency rule: the language can be playful at the source level, but the execution route must remain inspectable. The UI thus resembles a translation view more than a black-box compiler. That choice follows the Joelib translator's dual-panel, directional design and the Joelements project's paired original/Joe naming views.[3, 6]

The normative initial implementation is a browser-hosted console. It requires no server-side compilation for basic programs, stores only optional local history, and runs projected code in an isolated worker or iframe sandbox. Network access, document access, and unrestricted dynamic imports are outside the base language profile.

6 Core Language Rules

6.1 The Joebject rule

Every program-level unit of behavior belongs to a Joebject, except a small set of console directives. A Joebject has a unique name in its module and may contain state declared with `has`, behavior declared with `does`, and optional event handlers declared with `when`.

```
joebject Lantern {
  has charge = 3

  does glow() {
    show "The lantern glows."
    give charge
  }
}
```

6.2 The receiver rule

A state-changing behavior must have an explicit or inferable receiver. Within a Joebject method, the current Joebject is the default receiver. Outside a method, a message must identify its receiver. This rule makes mutation legible and limits accidental writes to anonymous global state.

6.3 The state rule

State begins with `has`. A state name may be read in its owning behavior without a receiver prefix. A state update is valid only inside that owner or through a declared behavior. Direct hidden-field mutation is not part of the base profile.

6.4 The message rule

Behavior is invoked as a message: a receiver followed by a behavior name and ordered arguments. The parser rejects a message to an unknown receiver or an unknown behavior before host code is emitted.

6.5 The projection rule

Source transformation must never alter text inside string literals, comments, numeric literals, or literal property keys unless the author explicitly requests a display-only Joeglish rendering. This avoids the common error of turning program data into syntax.

7 Normative Grammar

The following compact grammar uses extended Backus–Naur form. Braces mean repetition, brackets mean an optional term, and quoted words are reserved terminals.

```
program       = { directive | joebject-decl | message-stmt } ;
directive     = import-directive | display-directive ;
joebject-decl = "joebject" identifier "{" { member } "}" ;
member        = state-decl | behavior-decl | event-decl ;
state-decl    = "has" identifier "=" expression ;
behavior-decl = "does" identifier "(" [ parameters ] ")" block ;
event-decl    = "when" expression block ;
block         = "{" { statement } "}" ;
statement     = state-update | message-stmt | conditional |
                "show" expression | "give" expression ;
message-stmt  = expression "." identifier "(" [ arguments ] ")" ;
conditional   = "when" expression block [ "otherwise" block ] ;
import-directive = "ask" identifier "from" string ;
```

Reserved keywords are: `joebject`, `has`, `does`, `when`, `otherwise`, `give`, `show`, `ask`, `from`, `each`, `in`, `newjoe`, `truejoe`, `falsejoe`, and `nonejoe`. The first implementation may add library names, but it must not silently reassign a published keyword.

Identifiers should be readable ordinary words or Joeglish-compatible words. The compiler preserves a user-selected display spelling, while a normalized internal identifier is generated for collision avoidance. The dictionary process described below makes this normalization reversible whenever an identifier is fully dictionary-backed.

8 Central Algorithms

8.1 Algorithm A: lexical projection

Let S be a source sequence. The lexer returns tokens $T = (t_1, \ldots, t_n)$, each with a class $c(t_i)$. Let M be a validated partial bijection between English and Joeglish display words, and let K be the set of reserved Joe tokens. The forward display projection P is:

$$P(t_i) = \begin{cases} t_i & c(t_i) \in \{\text{string}, \text{comment}, \text{number}, \text{punctuation}\}, \\ t_i & t_i \in K, \\ M(t_i) & t_i \in \text{dom}(M) \text{ and display projection is enabled}, \\ t_i & \text{otherwise.} \end{cases}$$

The inverse uses M^{-1}. A projection is accepted as round-trippable when $M^{-1}(P(T)) = T$ after whitespace normalization. The Joeglish generator provides the precedent for checking one-to-one correspondence, excluding collisions, rejecting a generated form equal to its source, and testing reverse lookup.[4]

8.2 Algorithm B: Joebject normalization

For each parsed Joebject J, construct a canonical tuple

$$N(J) = (\text{name}, \text{stateMap}, \text{behaviorMap}, \text{events}, \text{imports}).$$

The normalizer checks that state names are unique, behavior names are unique at their declared arity, and every reference inside a behavior resolves either to local state, a parameter, an imported symbol, or an explicit receiver. Any unresolved name is a compile-time diagnostic, not an implicit global.

8.3 Algorithm C: receiver-first message routing

For a message expression $r.m(a_1, \ldots, a_q)$, the compiler resolves r before m. If r resolves to a Joebject instance and m exists in its behavior map, the generator emits the equivalent host method call. If either stage fails, it emits no executable host call and reports the closest known receiver or behavior. Resolution is $O(1)$ average time using maps; diagnostic suggestion may add $O(k)$ work for k candidate names.

8.4 Algorithm D: safe host emission

Each Joebject becomes either a JavaScript `class` or a closure-backed object record. State declarations become owned fields; behaviors become methods; `show` becomes a console output bridge; `give` becomes a return statement; and `ask` becomes an approved module binding. Generated code includes source-span markers so a host exception can be mapped back to the authored Joe source.

9 Compiler and Console Embodiment

An online implementation contains five visible panes or modes:

1. **Source**: editable JoeLang text with keyword highlighting;
2. **Projection**: generated JavaScript with mappings exposed rather than concealed;
3. **Console**: output, returned values, warnings, and errors;
4. **Joebject Inspector**: a tree of runtime objects, state, and methods; and
5. **Archive**: contextual reference cards that link Joe vocabulary to Joelib source materials.

The display may borrow the disciplined duality of translation tools: one side is the text a human chose to write, and the other is the corresponding representation. The console is not merely decoration. It implements a verification procedure. On each run it records the source hash, the normalized Joebject graph, the projection version, the generated-code hash, and the result category. A user can therefore reproduce a diagnostic by preserving a single console record.

The console should provide a "show the Joe" command that highlights the path from a selected source token to its generated code. For a dictionary-derived identifier, the UI displays original word, Joeglish display form, mapping source, and reverse check. For a structural keyword, it displays the grammar production that accepted it. For a runtime field, it displays the owning Joebject.

10 Library and Archive Model

The base library is named **Joelib** inside the language but does not overwrite or distribute the original repository. Its modules are called **Joelements**: stable host capabilities that receive Joe-facing names, documentation, and examples. A Joelement must state its host capability, allowed inputs, output shape, and whether it can alter state.

Examples of initial module categories are:

- `Joerbon`: text values, joining, display-safe formatting, and word operations;
- `Joexyjin`: timers, scheduled messages, and event signals;
- `Joerine`: approved local storage and serialization;
- `Joekel`: numeric utilities and validation;
- regional **Joekemon packages**: optional collections grouped by catalog region, with each package presenting a small, inspectable utility set.

The names above are presentation-layer conventions, not assertions that chemistry or the source records dictate a computing API. The Joelements project demonstrates the relevant design pattern: an original identity remains present as an anchor while the Joe name becomes the foreground label.[6] Likewise, the Joekemon TSV files provide a useful catalog structure in which regions can organize packages without requiring a user to memorize a single flat namespace.[7]

11 Diagnostics, Safety, and Determinism

11.1 Diagnostic classes

The compiler emits four ordered classes of findings: lexical, structural, projection, and runtime. Examples include “Unknown Joebject,” “Behavior not observed on receiver,” “State may only change through its owner,” and “Projection mapping is not reversible.” Messages must include a source span, a plain-language explanation, and one action a user can take.

11.2 Safety profile

The base online console must execute generated code inside a restricted sandbox. It does not expose browser documents, network requests, host filesystem access, arbitrary module URLs, or parent-window objects. Console output is copied as text. A later, explicitly authorized extension may add selected capabilities through Joelement modules, each of which must expose permission state before a program runs.

11.3 Determinism profile

Given the same language version, dictionary version, source text, approved module versions, and input values, projection output must be byte-stable except for source-map identifiers. Randomness and current time are supplied only through explicit Joelements so they can be recorded, seeded, or mocked.

11.4 Accessibility profile

All console panes must be keyboard reachable. Syntax color cannot be the sole carrier of meaning. Projection and diagnostics must have textual alternatives, and a user must be able to copy generated code and the diagnostic record without relying on drag-and-drop interactions.

12 Illustrative Claims

The following claims describe the intended scope of the internal technical proposal. They are explanatory rather than legal claims.

Claim 1. A computer-implemented method comprising receiving object-facing source text; parsing the text into Joebject declarations; normalizing each Joebject into state and behavior maps; projecting the normalized representation into host code; executing the host code in a sandbox; and presenting source, projection, and result in a common console.

Claim 2. The method of Claim 1, wherein the source text includes a lexical display mapping governed by a partial bijection whose inverse is verified before a reversible display projection is reported as valid.

Claim 3. The method of Claim 1, wherein an operation that changes state is accepted only when an owning Joebject receiver is explicit or inferable from the enclosing behavior.

Claim 4. The method of Claim 1, wherein string literals, comments, numeric literals, and punctuation are excluded from lexical display projection.

Claim 5. The method of Claim 1, wherein the console visualizes a correspondence between a source token and a generated host-code span.

Claim 6. A programming language having reserved declarations for a Joebject, owned state, behavior, message routing, conditional behavior, return values, display output, and approved module acquisition.

Claim 7. The language of Claim 6, wherein package modules are documented as Joelements while retaining a stated host-level capability and permission surface.

Claim 8. The language of Claim 6, wherein an unresolved symbol produces a compile-time diagnostic rather than an implicit global host-language binding.

Claim 9. A browser-based development environment that renders authored source, generated host code, runtime output, and Joebject state in separate but linked panels.

Claim 10. The environment of Claim 9, wherein execution is denied capabilities outside an approved module list.

13 Conclusion and Use Notice

JoeLangis specified as a transparent object-facing language whose source vocabulary is informed by Joelib's documented Joe universe. Its new contributions are the Joebject discipline, receiver-first routing, safe lexical projection, inspectable JavaScript emission, and a console that keeps authoring and execution visibly connected. The specification preserves the distinction between the archive materials and the newly proposed language rules.

This closing notice makes the framing explicit: this patent-style disclosure is a parody and creative design artifact, not a legal patent, a claim of discovery, or a replacement for the cited Joelib works. Source materials remain credited to their named contributors and to the repository record.

References

[1] N0TD3R3K/Joelib contributors. *Joelib README*. Repository index to the Joe Wiki, Joeglish Grammar, *The Great Joetsby*, Joelements, Joekemons, Joelet Speech, JOESII, and Joegle Translate. Available at https://github.com/N0TD3R3K/Joelib/blob/main/README.md. Accessed 2026-11-06.

[2] Derek Li (attribution supplied by the Joe Team). *Joelib Wiki: Home*. Joe-universe introduction, discovery record, and science/culture/literature overview. Available at https://github.com/N0TD3R3K/Joelib/wiki. Accessed 2026-11-06.

[3] Wayne Xu (attribution supplied by the Joe Team). *Joegle Translate README*. Description of the English–Joeglish interface, deterministic transformation, round-trip property, and source vocabulary. Available at https://github.com/N0TD3R3K/Joelib/blob/main/joeglish_translator/README.md. Accessed 2026-11-06.

[4] Wayne Xu (attribution supplied by the Joe Team). *Joeglish dictionary generator*. Seed pairs, candidate transformation rules, collision checks, and bijectivity assertions. Available at https://github.com/N0TD3R3K/Joelib/blob/main/joeglish_translator/scripts/generateDictionary.js. Accessed 2026-11-06.

[5] Wayne Xu (attribution supplied by the Joe Team). *Joeglish translation engine*. Tokenization, casing preservation, directional lookup, and round-trip computation. Available at https://github.com/N0TD3R3K/Joelib/blob/main/joeglish_translator/src/engine/joeglishEngine.ts. Accessed 2026-11-06.

[6] Derek Li (attribution supplied by the Joe Team). *The Joeriodic Table of Joelements*, including interface and structured element dataset. Available at https://github.com/N0TD3R3K/Joelib/tree/main/joelements. Accessed 2026-11-06.

[7] Matthew DeRoma (attribution supplied by the Joe Team). *Joekemon regional TSV collections*. Available at https://github.com/N0TD3R3K/Joelib. Accessed 2026-11-06.

[8] Wayne Xu (attribution supplied by the Joe Team). *JOESII and JOESII art*. Canvas-based ASCII signal presentation. Available at https://github.com/N0TD3R3K/Joelib/tree/main/joesii. Accessed 2026-11-06.

[9] Emilia Marcucci (attribution supplied by the Joe Team). *Joelet Speech*. Repository PDF and README-linked document. Available at https://github.com/N0TD3R3K/Joelib/blob/main/Joelet%20Speech.pdf. Accessed 2026-11-06.

[10] Joe Jin (the central figure himself; attribution supplied by the Joe Team). *Joeglish Grammar*. Linked from the Joelib README. Available through https://github.com/N0TD3R3K/Joelib/blob/main/README.md. Accessed 2026-11-06.

[11] Derek Li (attribution supplied by the Joe Team). *The Great Joetsby*, linked from the Joelib README. Available through https://github.com/N0TD3R3K/Joelib/blob/main/README.md. Accessed 2026-11-06.

[12] N0TD3R3K/Joelib contributors. *Contributing Guidelines*. Statement inviting further Joe-universe studies and documents. Available at https://github.com/N0TD3R3K/Joelib/blob/main/CONTRIBUTING.md. Accessed 2026-11-06.